

MSDS 535

Chapter 4 – Week 4

Overview and Objectives



Understand basic concepts.



Interpret frequent itemset mining methods.



Compare and contrast patterns that are interesting.



Evaluate pattern evaluation methods.



Summarize Data basic concepts and methods.

Introduction to Pattern Mining

- Frequent patterns include itemsets, subsequences, or substructures that appear frequently in a dataset.
- Examples include frequent itemsets (e.g., milk and bread), frequent sequential patterns (e.g., smartphone, smart TV, smart home device), and frequent structured patterns (e.g., subgraphs, subtrees).
- Finding frequent patterns is essential for mining associations, correlations, and relationships within data.
- It aids in data classification, clustering, and other data mining tasks.
- Frequent pattern mining is a significant research focus in data mining.

Basic Concepts in Frequent Pattern Mining

- Frequent pattern mining uncovers recurring relationships in datasets.
- Market basket analysis is an example, focusing on associations and correlations between itemsets in transactional data.
- The process involves finding frequent itemsets to discover associations.
- Association rules ($A \Rightarrow B$) are derived from frequent itemsets, based on support and confidence measures.
- Support measures the occurrence of $A \cup B$, while confidence measures the probability of B given A .

Market Basket Analysis

- Market basket analysis explores customer buying habits through associations between items.
- It helps in catalog design, cross-marketing, and customer shopping behavior analysis.
- Retailers gain insights into which items are frequently purchased together by customers.
- Strategies like store layouts, selective marketing, and sales planning benefit from market basket analysis.

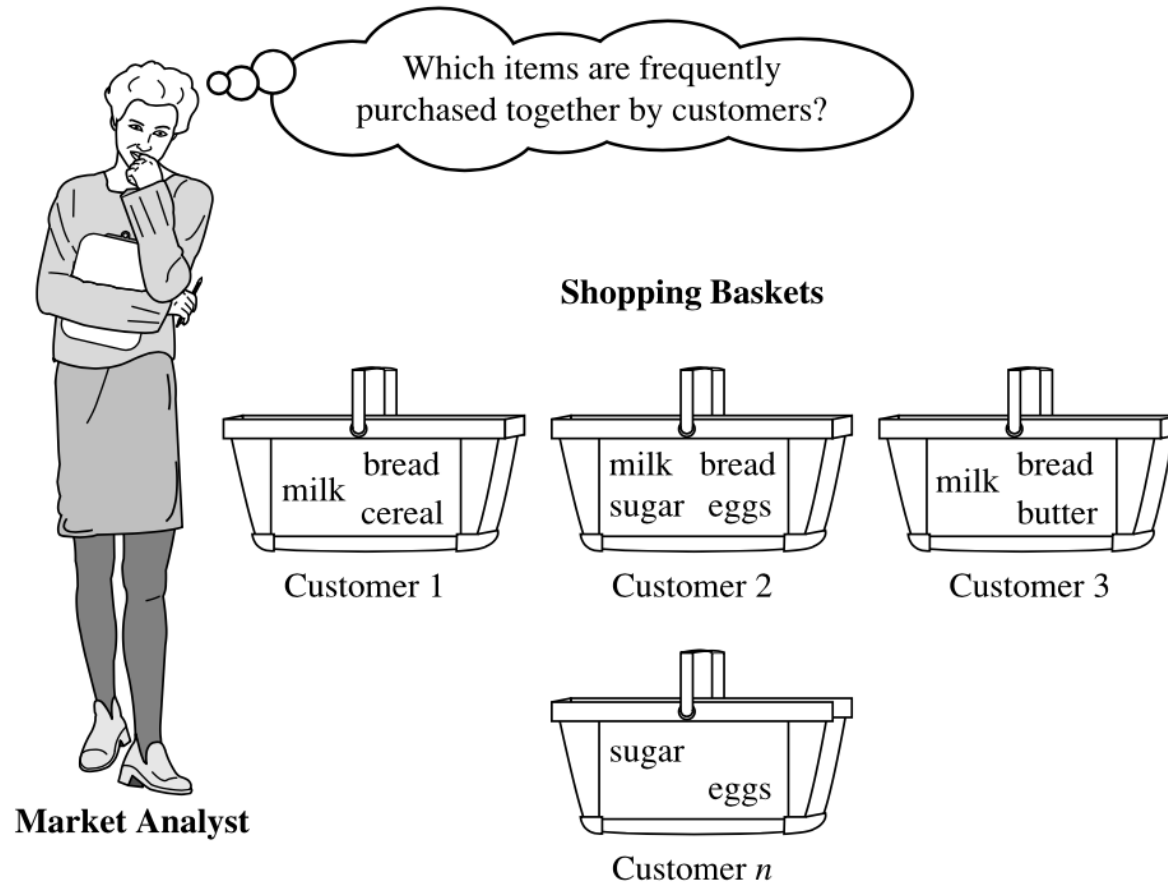


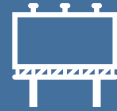
FIGURE 4.1

Market basket analysis.

Example of Market Basket Analysis



An example illustrates market basket analysis's practical utility.



Retail managers can use it to understand customer buying habits.



Strategies like store layout design and selective marketing are informed by analysis results.



Discounts and promotions can be tailored based on customer buying patterns.

Frequent Itemsets, Closed Itemsets, and Association Rules

- Association rules are in the form $A \Rightarrow B$, where A and B are non-empty itemsets.
- A transaction set D contains database transactions, where each transaction T is a non-empty itemset.
- Each transaction in D is associated with a unique identifier called a TID.
- A transaction T is considered to contain itemset A if A is a subset of T .

Frequent Itemsets, Closed Itemsets, and Association Rules

- Support (s) of a rule $A \Rightarrow B$ is the percentage of transactions in D containing both A and B.
- Confidence (c) of a rule $A \Rightarrow B$ is the percentage of transactions containing A that also contain B.
- Strong rules meet both a minimum support threshold (min_sup) and a minimum confidence threshold (min_conf).
- Support and confidence values are typically represented as percentages.

Frequent Itemsets, Closed Itemsets, and Association Rules



A k -itemset contains k items from a given itemset I .



The occurrence frequency of an itemset refers to the number of transactions that contain it and is also known as frequency, support count, or count.



Frequent itemsets are those whose relative support satisfies a predefined minimum support threshold (min_sup), and they are often denoted as L_k .



Confidence in an association rule $A \Rightarrow B$ can be calculated using the support counts of A , B , and $A \cup B$.

Frequent Itemsets, Closed Itemsets, and Association Rules

- Association rule mining involves two main steps:
- Find all frequent itemsets, each satisfying a minimum support count threshold (min_sup).
- Generate strong association rules from the frequent itemsets, which must meet minimum support and minimum confidence criteria.
- Closed frequent itemsets are those with no proper superitemset having the same support count.

Frequent Itemsets, Closed Itemsets, and Association Rules



Maximal frequent itemsets are frequent itemsets for which no superitemset is frequent.



The set of closed frequent itemsets (C) and maximal frequent itemsets (M) contain information about frequent itemsets, with C providing complete support count information.



Mining frequent itemsets can lead to a vast number of itemsets, especially when `min_sup` is set low, which is computationally challenging.



To address this issue, closed frequent itemsets and maximal frequent itemsets are introduced to efficiently represent and manage frequent itemset information.

Apriori Algorithm and Its Components

- Apriori is a significant algorithm for mining frequent itemsets in Boolean association rule mining, introduced by R. Agrawal and R. Srikant in 1994.
- It utilizes a level-wise search approach, starting with the discovery of frequent 1-itemsets (L_1) by scanning the database and accumulating counts for each item that satisfies the minimum support.
- The algorithm progresses to find frequent k -itemsets (L_k) by building on the results of the previous level (e.g., L_2 is derived from L_1).
- The Apriori property is employed to optimize the level-wise generation of frequent itemsets, which states that all nonempty subsets of a frequent itemset must also be frequent.

Apriori Algorithm and Its Components

- The Apriori property is based on antimonotonicity, meaning that if a set doesn't pass a test, all of its supersets will fail the same test.
- The algorithm's core process for finding L_k for $k \geq 2$ involves a two-step approach: join and prune.
- The join step involves generating candidate k -itemsets (C_k) by joining L_{k-1} with itself. Items are sorted in lexicographic order for efficient implementation.
- The prune step reduces the size of C_k using the Apriori property. If any $(k - 1)$ -subset of a candidate k -itemset is not in L_{k-1} , the candidate is not frequent and can be removed from C_k .
- Maintaining a hash tree of all frequent itemsets enables quick subset testing to determine whether candidates in C_k are frequent.

Generating Association Rules

- After identifying frequent itemsets in a database D , strong association rules can be generated from them by satisfying both minimum support and minimum confidence.
- Confidence ($A \Rightarrow B$) is determined using $\text{support_count}(A \cup B)$ and $\text{support_count}(A)$, where $\text{support_count}(A \cup B)$ is the number of transactions containing the itemsets $A \cup B$, and $\text{support_count}(A)$ is the number of transactions containing the itemset A .

Generating Association Rules

- The process for generating association rules is as follows:
- For each frequent itemset l , generate all nonempty subsets of l .
- For every nonempty subset s of l , output the rule " $s \Rightarrow (l - s)$ " if $\text{support_count}(l) \geq \text{min_conf}$, where min_conf is the minimum confidence threshold.
- The Apriori algorithm, which employs an iterative level-wise approach based on candidate generation, is used for finding frequent itemsets in D .

Generating Association Rules

- Input for Apriori:
- D: a database of transactions.
- min_sup: the minimum support count threshold.
- Output: L, which contains frequent itemsets in D.

Generating Association Rules

- Apriori algorithm steps:
 1. Find frequent 1-itemsets (L_1) by scanning D .
 2. Iterate for $k = 2$ to until L_{k-1} is not empty:
 - a. Generate candidate k -itemsets (C_k) using `apriori_gen`.
 - b. For each transaction t in D , scan D for counts and obtain subsets of C_k that are candidates for each candidate c in C_t .
 - c. Update the count for each candidate c .
 - d. L_k contains candidates with counts $\geq \text{min_sup}$.
 3. Return L , which is the union of all L_k .

Generating Association Rules

- The procedure `apriori_gen` is used to generate candidates for k-itemsets based on the Apriori property.
- The procedure `has_infrequent_subset` checks if a candidate k-itemset has infrequent subsets based on prior knowledge.
- Frequent itemsets can be stored in hash tables along with their counts for quick access when generating association rules.

Improving Apriori Efficiency

- Several variations of the Apriori algorithm have been proposed to improve its efficiency for mining frequent itemsets:
- Hash-based technique: This technique reduces the size of candidate k -itemsets (C_k) for $k > 1$ by hashing itemsets into corresponding buckets, allowing the removal of itemsets with counts below the support threshold.
- Transaction reduction: Transactions that do not contain frequent k -itemsets are marked or removed from further consideration, reducing the number of transactions scanned in future iterations.

Improving Apriori Efficiency

- Partitioning: Partitioning divides the database transactions into non-overlapping partitions, with local frequent itemsets found within each partition. These local frequent itemsets become candidate itemsets with respect to the entire database.
- Sampling: The sampling approach involves mining frequent itemsets in a random sample (S) of the given data D , reducing the computational load. Lower support thresholds are used in S , and a mechanism checks if all global frequent itemsets are included in LS . A second pass may be done to find missed frequent itemsets.
- Dynamic itemset counting: This technique involves adding candidate itemsets at different points during a scan, unlike Apriori, which determines new candidate itemsets only after each complete database scan. The count-so-far is used as a lower bound of the actual count, leading to fewer database scans than Apriori.

Partitioning

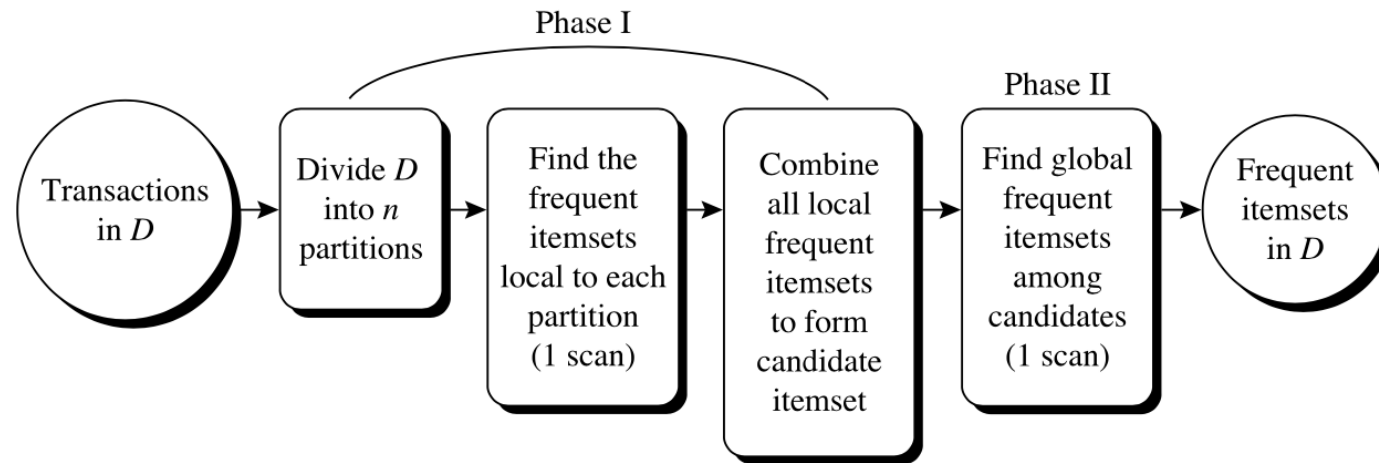


FIGURE 4.6

Mining by partitioning the data.

Pattern-Growth Approach and Vertical Data Format

- Frequent Pattern Growth (FP-growth) is an alternative method for mining frequent itemsets that aims to eliminate the need for a costly candidate generation process seen in Apriori-based methods.

Pattern-Growth Approach and Vertical Data Format

- FP-growth employs a divide-and-conquer strategy that involves the following steps:
 1. The first database scan identifies frequent 1-itemsets and their support counts.
 2. An FP-tree (Frequent Pattern Tree) is constructed to represent the frequent items' associations, which forms the compressed database.
 3. The FP-tree is divided into a set of conditional databases, each associated with a frequent itemset found so far.
 4. Mining is performed on each conditional database separately.

Pattern-Growth Approach and Vertical Data Format

- FP-tree construction: During the second scan of the database, transactions are processed in descending support count order.
- An FP-tree is constructed by creating branches for each transaction, and nodes are generated for the items in each transaction.
- Nodes along common prefixes are shared, reducing space requirements.
- Item Header Table: To facilitate tree traversal, an item header table is built, enabling each item to point to its occurrences in the FP-tree via node-links.

Pattern-Growth Approach and Vertical Data Format

- Mining the FP-tree: Mining begins with frequent length-1 patterns as initial suffix patterns.
- The process involves constructing a conditional pattern base for each frequent item and building a conditional FP-tree.
- Mining is then performed recursively on the conditional FP-tree.
- The pattern growth is achieved by concatenating the suffix pattern with patterns generated from the conditional FP-tree.

Pattern-Growth Approach and Vertical Data Format

- Efficiency: FP-growth aims to find long frequent patterns by searching for shorter ones in smaller conditional databases recursively.
- It starts with the least frequent items as suffixes, offering better selectivity and significantly reducing search costs.

Pattern-Growth Approach and Vertical Data Format

- Partitioning: In cases where the FP-tree cannot fit into main memory for large databases, partitioning the database into a set of projected databases is an alternative approach.
 - This recursive partitioning can be applied until each projected database's FP-tree fits in main memory.
- Advantages: FP-growth reduces candidate generation and database scanning costs compared to Apriori, making it efficient for mining frequent itemsets, especially in large databases.

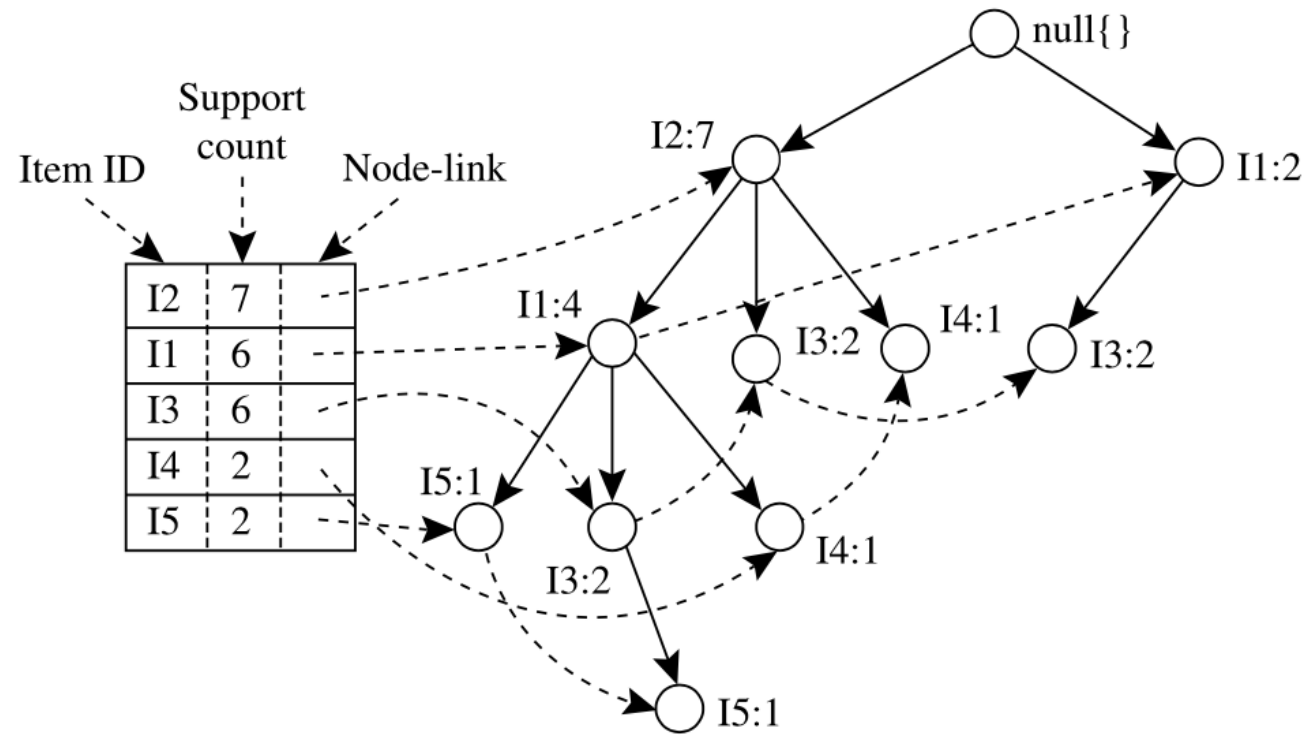


FIGURE 4.7

An FP-tree registers compressed frequent pattern information.

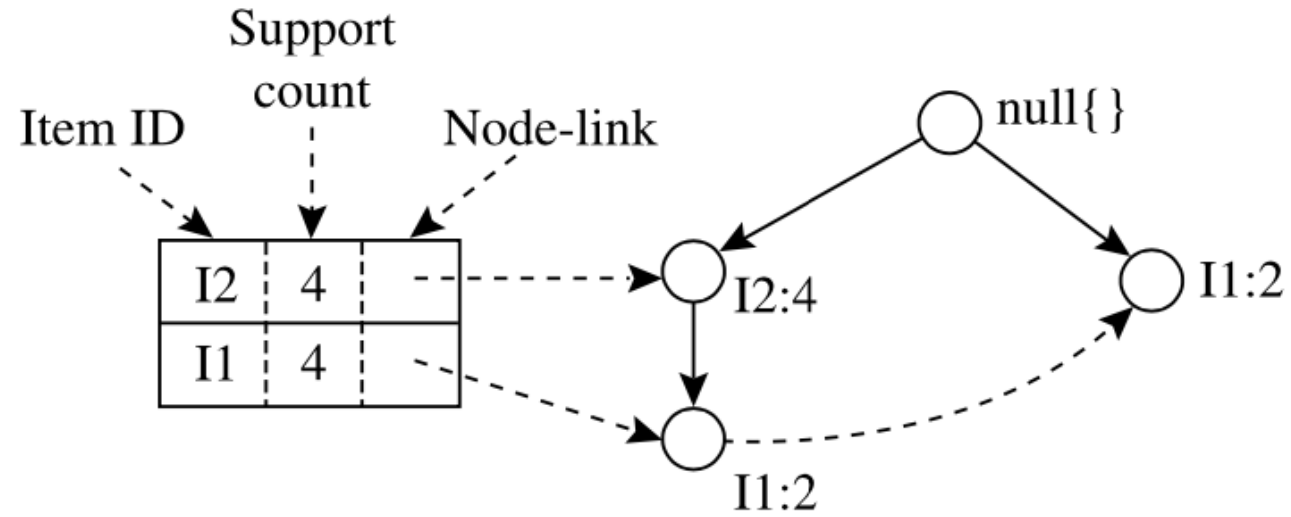


FIGURE 4.9

The conditional FP-tree associated with the conditional node I3.

Pattern-Growth Approach and Vertical Data Format

- The vertical data format is an efficient alternative to horizontal format for mining frequent itemsets.
- Eclat (Equivalence Class Transformation) algorithm leverages the vertical format by intersecting TID_sets to find frequent itemsets.

Mining Closed and Max Patterns

- Frequent itemset mining generates numerous patterns, especially with low support thresholds and long patterns.
- Closed frequent itemsets reduce the number of patterns while preserving all information about frequent itemsets.

Mining Closed and Max Patterns

- It is more practical to mine closed frequent itemsets rather than all frequent itemsets in most cases.
- Mining closed frequent itemsets requires methods to efficiently identify them during the mining process.
- Pruning techniques, like itemset merging, help in identifying and mining closed itemsets efficiently.

Pattern Evaluation Methods - Overview

- Association rule mining algorithms typically use a support-confidence framework to filter rules.
- Even strong association rules may not be interesting to users in certain cases, especially with low support or long patterns.
- This section introduces various pattern evaluation methods to address the issue of uninteresting rules.

Pattern Evaluation Methods - Overview

- These methods include support, confidence, lift, χ^2 , all_confidence, max_confidence, Kulczynski, and cosine measures.
- Researchers have explored these measures to identify truly interesting patterns and discard uninteresting ones.

Strength of Association Rules

- Strong association rules may not always be interesting to users, leading to the need for objective interestingness measures.
- Confidence can be misleading in determining the real strength of a correlation.
- Objective measures, based on data statistics, can help filter out uninteresting rules.
- Lift is introduced as a correlation measure that assesses the dependency between itemsets.
- The lift measure helps avoid misinterpretations of confidence.

Correlation Analysis

- Correlation measures, like lift and χ^2 , are discussed as supplements to the support-confidence framework.
- The lift measure quantifies the extent to which the occurrence of one itemset influences the other.
- Lift values greater than 1 indicate positive correlation, less than 1 indicate negative correlation, and 1 indicates independence.

Correlation Analysis

- χ^2 measures the deviation of observed and expected values in a contingency table and is another correlation measure.
- Both measures are sensitive to the number of null-transactions and can yield unstable results in certain scenarios.
- There is a need for null-invariant measures, independent of null-transactions

All_Confidence

- Defined as the minimum of conditional probabilities $P(A | B)$ and $P(B | A)$.
- Represents the minimum confidence of the two association rules " $A \Rightarrow B$ " and " $B \Rightarrow A$."
- Ranges from 0 to 1, with higher values indicating a stronger relationship between A and B.
- Independent of the total number of transactions.

Max_Confidence

- Defined as the maximum of conditional probabilities $P(A | B)$ and $P(B | A)$.
- Represents the maximum confidence of the two association rules " $A \Rightarrow B$ " and " $B \Rightarrow A$."
- Ranges from 0 to 1, with higher values indicating a stronger relationship between A and B.
- Independent of the total number of transactions.

Kulczynski (Kulc)

- Defined as $1/2$ times the sum of conditional probabilities $P(A | B)$ and $P(B | A)$.
- Introduced by mathematician S. Kulczynski.
- Represents an average of the two conditional probabilities, indicating the overall relationship between A and B.
- Ranges from 0 to 1, with higher values indicating a stronger relationship between A and B.
- Independent of the total number of transactions.

Cosine

- Defined as the square root of the product of conditional probabilities $P(A | B)$ and $P(B | A)$.
- Can be viewed as a harmonized lift measure, but it uses the square root.
- Influenced only by the supports of A , B , and $A \cup B$, not by the total number of transactions.
- Ranges from 0 to 1, with higher values indicating a stronger relationship between A and B .

Imbalance Ratio (IR)

- The topic is the "imbalance ratio (IR)" used to assess the imbalance between two itemsets, A and B, in rule implications.
- The purpose of IR is to quantify the imbalance between A and B in terms of support in transactions.
- If the two directional implications between A and B are the same, $IR(A,B)$ will be zero.
- A larger difference between the support of A and B results in a larger imbalance ratio.
- Importantly, the imbalance ratio is independent of the number of null-transactions and also independent of the total number of transactions.

Comparison of Pattern Evaluation Measures

- Results show that null-invariant measures are better at distinguishing interesting pattern relationships.
- Kulczynski and IR together provide a clear picture of pattern relationships in scenarios with imbalanced data.

Summary

- Frequent pattern mining is valuable for applications like market basket analysis.
- Association rule mining involves finding frequent itemsets and generating strong association rules.
- Efficient algorithms for frequent itemset mining include Apriori-like algorithms, frequent pattern growth, and vertical data format mining (Eclat).

Summary

- Apriori algorithm uses candidate generation and scanning while maintaining the Apriori property.
- Frequent pattern growth constructs an FP-tree and avoids costly candidate generation.
- Vertical data format mining transforms data and intersects item-TID sets.
- Pattern evaluation measures help identify interesting association rules.

Summary

- Null-invariant measures, unaffected by null-transactions, are preferable.
- Lift, χ^2 , all_confidence, max_confidence, Kulczynski, and cosine measures were compared, with only the last four being null-invariant.
- The Kulczynski measure, combined with the imbalance ratio, is recommended for presenting pattern relationships.